Agentes Inteligentes

Definición de un problema para Agentes Inteligentes

Estudiante: Constanza Flores Vásquez
Profesor: Mauricio Araya
Fecha: Agosto 2026

---

1. **¿Qué problema se desea resolver?**

   - **Tetris:** Puzzle individual donde piezas geométricas (tetraminós) caen en una cuadrícula de 10×20. Al completar filas horizontales, estas se eliminan otorgando puntos; la partida termina al sobrepasar el borde superior de la cuadrícula. Es interesante porque el agente debe optimizar la limpieza de líneas sin saturar el tablero, esto bajo un flujo estocástico de piezas.
   - **Pokémon TCG:** Juego de cartas 1v1 por turnos donde se gestionan criaturas, energías y soporte para noquear al rival. Es interesante por la toma de decisiones bajo información que desconocemos (mano rival y mazo ocultos) y gestión de energía para maximizar las oportunidades de ganar en un futuro.
   - **Quoridor:** Juego de tablero 9×9 para dos jugadores donde se elige entre mover el peón o colocar vallas para alargar la ruta del oponente. Es interesante porque combina búsqueda de caminos con un grafo dinámico y un amplio factor de ramificación.

2. **¿Qué problema se selecciona y por qué?**
   Se selecciona **Tetris**. A pesar de sus reglas simples, presenta un espacio de estados inmenso, donde malas decisiones pueden generar consecuencias graves a largo plazo. Es un entorno determinista en su física pero estocástico en la llegada de piezas, con métricas claras y cuantificables en tiempo real (líneas, supervivencia). Además, es factible con los contenidos del curso y escalable agregándole dificultad como velocidad a las piezas o conocer la pieza siguiente (n+1). Al tratarse de un problema de tipo NP-completo, resolver la partida perfecta a largo plazo resulta computacionalmente intratable. Por ello, para términos de este proyecto se aborda una versión simplificada y orientada a la búsqueda local: en lugar de planificar sobre secuencias infinitas de piezas, el agente toma decisiones reactivas turno a turno evaluando un conjunto acotado de heurísticas del tablero con una cantidad de pieza fijas.

3. **¿Quién o qué es el agente y cuál es su objetivo?**
   El agente es un software autónomo que asume el rol del jugador. Su función consiste en percibir el estado de la matriz, evaluar las posibles colocaciones de la pieza activa y ejecutar la mejor jugada.

El objetivo principal del agente es maximizar la supervivencia y la cantidad de líneas eliminadas dentro de un límite establecido de piezas por partida (por ejemplo, 1.000 piezas), evitando que la pila de bloques alcance el límite superior del tablero. Se considerará que el objetivo se ha logrado si el agente completa la secuencia total de piezas sin llegar al estado de pérdida (*Game Over*); como criterio complementario de desempeño, se busca maximizar la puntuación acumulada para verificar que el sistema refine su estrategia en cada iteración.

4. **¿Cómo interactúa el agente con su ambiente?**

   - **Sensores (Entrada):** Lee el estado actual de la matriz binaria del tablero (10×20) y la información geométrica (tipo y orientación inicial) de la pieza activa en turno.
   - **Decisión (Procesamiento):** Genera y evalúa, mediante la función heurística parametrizada, los tableros resultantes de todas las combinaciones válidas de rotación y columna para la pieza actual.
   - **Actuadores (Salida):** Ejecuta de forma directa la colocación en la posición óptima seleccionada mediante caída instantánea (*hard drop*), fijando el bloque y actualizando el tablero de inmediato.

5. **¿Cómo puede caracterizarse el ambiente del agente?** El entorno simplificado de Tetris se caracteriza bajo las siguientes propiedades:

   - **Completamente observable:** El agente tiene acceso total a la matriz del tablero (10×20) y a la pieza activa en cada turno, sin variables ocultas ni incertidumbre perceptual.
   - **Un solo agente:** El agente opera de forma individual en el entorno, sin competencia directa ni colaboración con otros agentes.
   - **Estocástico:** El orden de aparición de las piezas entrantes está determinado por un generador pseudoaleatorio, impidiendo predecir con certeza los estados a mediano y largo plazo.
   - **Secuencial:** Cada colocación de pieza altera de manera irreversible la configuración del tablero, condicionando directamente el espacio de búsqueda y las opciones de los turnos posteriores dentro de la cuota límite fijada.
   - **Estático:** Al modelarse como un sistema por turnos discretos, el tablero se congela mientras el agente evalúa la jugada, eliminando restricciones de velocidad o cálculo en tiempo real.
   - **Discreto:** Tanto el espacio de estados (matriz finita de celdas), como el espacio de acciones (rotaciones fijas de 90° y columnas enteras) y el avance del tiempo (por piezas colocadas) están compuestos por un número finito y cuantificable de elementos.

6. **¿Qué decisiones deberá tomar el agente y qué espacio de soluciones deberá explorar?**

Decisiones del agente:
En cada turno, cuando aparece una nueva pieza, el agente debe decidir la orientación (cuántas veces rotarla) y la columna horizontal del tablero donde la dejará caer.

Espacio de soluciones a explorar:

- **Espacio de jugadas en la partida:** Para cada pieza existen entre 10 y 35 posiciones finales válidas donde puede encajar. El agente evalúa este conjunto de opciones inmediatas para elegir la mejor.
- **Espacio de optimización del algoritmo:** Mediante métodos de optimización de la unidad, el agente explora diversas combinaciones de prioridades o niveles de importancia asignados a las características del tablero (altura de la pila, cantidad de huecos generados, regularidad de la superficie y líneas eliminadas) hasta encontrar la configuración que le permita sobrevivir y sumar la mayor cantidad de puntos.

7. **¿Cómo podría abordarse inicialmente utilizando los métodos de la Unidad 1?**
El problema se modela como una optimización de caja negra sobre un entorno simplificado por turnos y con un límite de 1.000 piezas. Al no existir una fórmula analítica directa, la aptitud (*fitness*) de cada estrategia se evalúa empíricamente mediante la supervivencia y las líneas completadas en partidas de prueba.

   Para resolverlo, se combinan las siguientes técnicas:

   - **Búsqueda informada (por turno):** Evalúa en un solo paso ($lookahead = 1$) las posiciones legales de la pieza activa según criterios inmediatos: altura, huecos y líneas limpiadas.
   - **Búsqueda local y metaheurísticas:** Algoritmos como *Simulated Annealing* o *Tabu Search* optimizan los pesos de dichos criterios, permitiendo escapar de óptimos locales donde un *Hill Climbing* básico se estancaría.
   - **Algoritmos evolutivos:** Como alternativa, evolucionan una población de vectores de decisión mediante selección, cruce y mutación para explorar múltiples estrategias en paralelo.

8. **¿Cómo podría evolucionar el problema durante el curso?**

   - **Previsión (*Lookahead*):** Expandir el árbol de búsqueda para evaluar combinaciones de la pieza actual junto con la siguiente pieza en cola ($N = 2$).
   - **Cambio de paradigma:** Sustituir la búsqueda local de pesos fijos por Aprendizaje por Refuerzo (*Q-Learning* o *DQN*) para que el agente aprenda una política de control interactuando con el entorno.
   - **Condiciones dinámicas:** Incorporar velocidad de caída incremental para forzar al agente a operar bajo restricciones de tiempo real.

Referencias

[1] S. Algorta y Ö. Şimşek, "The Game of Tetris in Machine Learning", en *Proceedings of the Video Games and Machine Learning Workshop at the 34th International Conference on Machine Learning (ICML 2017)*, Sydney, Australia, 2017.

[2] E. D. Demaine, S. Hohenberger, y D. Liben-Nowell, "Tetris is Hard, Even to Approximate", en *International Colloquium on Automata, Languages, and Programming (ICALP 2003)*, ser. Lecture Notes in Computer Science, vol. 2719, Berlin, Heidelberg: Springer, 2003, pp. 376–388.